BLOCKSEC

Security Audit Report for onchain-evm

Date: June 27, 2025 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction **1**

1.1 About Target Contracts . . . 1

1.2 Disclaimer . . . 1

1.3 Procedure of Auditing . . . 2

1.3.1 Security Issues . . . 2

1.3.2 Additional Recommendation . . . 2

1.4 Security Model . . . 3

Chapter 2 Findings **4**

2.1 Recommendation . . . 4

2.1.1 Improper approve amount in the function `_safeApprove()` . . . 4

2.1.2 Lack of allowance revocation after swap . . . 5

2.1.3 Inconsistent check between `BGWSwapV1Executor` and `BGWSwapV1NewExecutor` 6

2.2 Note . . . 7

2.2.1 Ensure that `order.amountIn` matches the actual amount paid by the user . 7

2.2.2 Backend verification . . . 8

2.2.3 Ensure the `orderId` to be unique . . . 8

2.2.4 Loss of deployer management capability after ownership renouncement . 9

2.2.5 Potential centralization risks . . . 10

2.2.6 Security audit assumptions on trusted executors . . . 10

2.2.7 Weird ERC20 tokens . . . 10

2.2.8 The previous owner still retains deployer privileges after ownership is transferred . . . 10

2.2.9 Avoid mis-operations when deploying contract `DeployFactory` . . . 11

Report Manifest

Item	Description
Client	Bitget
Target	onchain-evm

Version History

Version	Date	Description
1.0	June 27, 2025	First release

Signature

Digitally signed by: BlockSec (https://blocksec.com)

SHA1 digest of public key: BAB106D3708BC6DE0963CE05E9478C24B326AE1C

Date: 2026-03-30 03:44:35+0000

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at Email, Twitter and Medium.

Chapter 1 Introduction

1.1 About Target Contracts

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The target of this audit is the code repository [1] of onchain-evm of Bitget.

Onchain-evm protocol is designed to process token swap orders from verified callers. It comprises the contract OnChainProxy, which serves as the entry point for verified callers, and multiple executor contracts(i.e., BGWSwapV1Executor, BGWSwapV1NewExecutor, and OneInchV6Executor), which interact with external routers to execute swaps.

Note this audit only focuses on the smart contracts in the following directories/files:

- src/*

Other files are not within the scope of the audit. Additionally, all dependencies of the smart contracts within the audit scope are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version (`Version 1`), as well as new code (in the following versions) to fix issues in the audit report.

Project	Version	Commit Hash
onchain-evm	`Version 1`	`45d0e5c27db925e7917054f6db46a646b966c8d6`
	`Version 2`	`c7d3331c4d3c9476d337612c7151d6871198aade`

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the smart contracts, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit can-

[1] https://github.com/blcokchainbg/onchain-evm

not be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of smart contracts.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan smart contracts with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of smart contracts and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

* Access control
* Permission management
* Whitelist and blacklist mechanisms
* Initialization consistency
* Improper use of the proxy system
* Reentrancy
* Denial of Service (DoS)
* Untrusted external call and control flow
* Exception handling
* Data handling and flow
* Events operation
* Error-prone randomness
* Oracle security
* Business logic correctness
* Semantic and functional consistency
* Emergency mechanism
* Economic and incentive impact

1.3.2 Additional Recommendation

* Gas optimization
* Code quality and style

Note *The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.*

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology [2] and Common Weakness Enumeration [3]. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact	Likelihood: *High*	Likelihood: *Low*
High	High	Medium
Low	Medium	Low

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

[2] https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

[3] https://cwe.mitre.org/

Chapter 2 Findings

In total, we have **three** recommendations and **nine** notes.

- Recommendation: 3
- Note: 9

ID	Severity	Description	Category	Status
1	-	Improper approve amount in the function `_safeApprove()`	Recommendation	Confirmed
2	-	Lack of allowance revocation after swap	Recommendation	Confirmed
3	-	Inconsistent check between `BGWSwapV1Executor` and `BGWSwapV1NewExecutor`	Recommendation	Fixed
4	-	Ensure that `order.amountIn` matches the actual amount paid by the user	Note	-
5	-	Backend verification	Note	-
6	-	Ensure the `orderId` to be unique	Note	-
7	-	Loss of deployer management capability after ownership renouncement	Note	-
8	-	Potential centralization risks	Note	-
9	-	Security audit assumptions on trusted executors	Note	-
10	-	Weird ERC20 tokens	Note	-
11	-	The previous owner still retains deployer privileges after ownership is transferred	Note	-
12	-	Avoid mis-operations when deploying contract `DeployFactory`	Note	-

The details are provided in the following sections.

2.1 Recommendation

2.1.1 Improper approve amount in the function `_safeApprove()`

Status Confirmed

Introduced by `Version 1`

Description The function `_safeApprove()` in the contract `ExecutorCommon` grants an unlimited token allowance (i.e., `type(uint256).max`) when the current allowance is insufficient. This practice can expose the contract to potential security risks if the `approveTarget` becomes compromised. It is recommended that only approves the strictly necessary amount.

```
function _safeApprove(address fromToken, address approveTarget, uint256 amountIn) internal {
    uint256 allowance = IERC20(fromToken).allowance(address(this), approveTarget);

    if (allowance < amountIn) {
        IERC20(fromToken).forceApprove(approveTarget, type(uint256).max);
    }
}
```

Listing 2.1: src/executors/ExecutorCommon.sol

Suggestion Only approve the strictly necessary amount when the current allowance is insufficient .

Feedback from the project The project stated that they had noticed it and decided to keep it for gas saving. They also stated that the executor contracts do not hold funds, thus the risk is low.

2.1.2 Lack of allowance revocation after swap

Status Confirmed

Introduced by `Version 1`

Description The contracts `BGWSwapV1Executor`, `BGWSwapV1NewExecutor` and `OneInchV6Executor` all implement swap functionality via the function `executeSwap()`, and the function `_safeApprove()` is called to grant allowance to the router. However, none of the implementations revokes the allowance after the swap operation completes. This creates a potential security risk when the router becomes compromised. It is recommended to implement allowance revocation logic after swap completion.

```
function executeSwap(ISwapExecutor.OrderInfo calldata order, bytes calldata data)
    external
    payable
    override
    onlyOnChainProxy
    nonReentrant
    whenNotPaused
{
    address fromToken = order.fromToken;
    uint256 amountIn = order.amountIn;

    if (_isETH(fromToken)) {
        if (msg.value != amountIn) revert InvalidETHAmount();
    } else {
        if (msg.value != 0) revert UnexpectedETHValue();
        _safeApprove(fromToken, router, amountIn);
    }

    _checkCalldata(order, data);

    (bool success,) = router.call{value: msg.value}(data);
    if (!success) revert CallFailed();

    _sweep(fromToken);
}
```

Listing 2.2: src/executors/BGWSwapV1Executor.sol

```
function executeSwap(ISwapExecutor.OrderInfo calldata order, bytes calldata data)
```

```
    external
    payable
    override
    onlyOnChainProxy
    nonReentrant
    whenNotPaused
{
    address fromToken = order.fromToken;
    uint256 amountIn = order.amountIn;

    if (_isETH(fromToken)) {
        if (msg.value != amountIn) revert InvalidETHAmount();
    } else {
        if (msg.value != 0) revert UnexpectedETHValue();
        _safeApprove(fromToken, router, amountIn);
    }

    _checkCalldata(data);

    (bool success,) = router.call{value: msg.value}(data);
    if (!success) revert CallFailed();

    _sweep(fromToken);
}
```

Listing 2.3: src/executors/OneInchV6Executor.sol

Suggestion Add allowance revocation logic after the swap operation.

Feedback from the project The project stated that they had noticed it and decided to keep it for gas saving. They also stated that the executor contracts do not hold funds, thus the risk is low.

2.1.3 Inconsistent check between `BGWSwapV1Executor` and `BGWSwapV1NewExecutor`

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `BGWSwapV1Executor`, the function `_isETH()` is overridden and only considers the zero address (i.e., `0x0000...000000`) as ETH. However, the function `_isETH()` in the contract `BGWSwapV1NewExecutor` directly inherits from contract `OnChainCommon`, which considers both the zero address and `0xEeee...eeEEeE` as ETH.

```
function _isETH(address tokenAddress) internal pure override returns (bool) {
    return tokenAddress == 0x0000000000000000000000000000000000000000;
}
```

Listing 2.4: src/executors/BGWSwapV1Executor.sol

```
function _isETH(address tokenAddress) internal pure virtual returns (bool) {
    return (tokenAddress == 0xEeeeeEeeeEeEeeEeEeEeeEEEeeeeEeeeeeeeEEeE) || (tokenAddress ==
        ZERO_ADDRESS);
}
```

Listing 2.5: src/OnChainCommon.sol

Suggestion Revise the logic accordingly.

2.2 Note

2.2.1 Ensure that `order.amountIn` matches the actual amount paid by the user

Introduced by `Version 1`

Description Currently, the contract `OnChainProxy` holds the funds, and a privileged caller account uses these funds to purchase the desired tokens on behalf of users based on orders generated by the backend. The backend service must perform proper validation to ensure that the user has actually paid the corresponding `order.amountIn` before executing the transaction.

```
function executeSwapByExecutor(ISwapExecutor.OrderInfo calldata order, bytes calldata data)
    external
    payable
    nonReentrant
    whenNotPaused
    onlyCaller
    validExecutor(order.executor)
    validRecipient(order.recipient)
    returns (uint256 exactAmountOut)
{
    bytes32 orderId = order.orderId;

    if (isOrderExecuted[orderId]) revert OrderAlreadyExecuted();
    isOrderExecuted[orderId] = true;

    _orderCheck(order);

    uint256 amountIn = order.amountIn;
    address executor = order.executor;
    address fromToken = order.fromToken;
    address toToken = order.toToken;
    address recipient = order.recipient;

    uint256 recipientBeforeBalance = _getBalance(toToken, recipient);

    uint256 inputAmount = 0;

    if (_isETH(fromToken)) {
        inputAmount = amountIn;
    } else {
        IERC20(fromToken).safeTransfer(executor, amountIn);
    }

    ISwapExecutor(executor).executeSwap{value: inputAmount}(order, data);
```

Listing 2.6: src/OnChainProxy.sol

Feedback from the project The project stated that the backend service will perform proper validation to ensure that the user has actually paid the corresponding `order.amountIn` before executing the transaction.

2.2.2 Backend verification

Introduced by `Version 1`

Description The contract `OnChainProxy` functions as a fund pool deployed on-chain in two identical instances. Specifically, one is used to store USDT while the other one is to store the other Tokens. The `caller`, `order`, and `data` are all server-controlled, where the `caller` exchanges USDT for the Tokens and sends them to the other OnChainProxy, and vice versa. Since users' specific order amounts and intentions are handled internally with the `caller` executing on their behalf, the contracts don't interact directly with end users. Consequently, all accounting and verification of user funds in this process must undergo backend validation to ensure security and accuracy.

For example, the `order` represents the user's intent (e.g., token swap details), while the `data` is the actual payload sent to the router for execution. Therefore, the backend must validate key parameters (such as `amountIn`, `recipient`, etc.) between the `order` and `data` to ensure that the executed transaction (`data`) strictly complies with the user's original intent (`order`).

Feedback from the project The project stated that the backend service will carefully validate the `data` and `order`.

2.2.3 Ensure the `orderId` to be unique

Introduced by `Version 1`

Description The function `executeSwapByExecutor()` in contract `OnChainProxy` checks for the same `order.orderId` through the mapping `isOrderExecuted`. This creates a potential vulnerability where the same `order.orderId` could cause transaction failures. The protocol should implement proper off-chain mechanisms to guarantee the value of `order.orderId` to be unique.

```
function executeSwapByExecutor(ISwapExecutor.OrderInfo calldata order, bytes calldata data)
    external
    payable
    nonReentrant
    whenNotPaused
    onlyCaller
    validExecutor(order.executor)
    validRecipient(order.recipient)
    returns (uint256 exactAmountOut)
{
    bytes32 orderId = order.orderId;

    if (isOrderExecuted[orderId]) revert OrderAlreadyExecuted();
    isOrderExecuted[orderId] = true;

    _orderCheck(order);
```

Listing 2.7: src/OnChainProxy.sol

Feedback from the project The project stated that it is guaranteed by the off-chain services.

2.2.4 Loss of deployer management capability after ownership renouncement

Introduced by `Version 1`

Description If the owner invokes the function `renounceOwnership()`, the contract `DeployFactory` will have no owner. In this state, the `manageDeployer()` functionality becomes disabled permanently, leaving deployer management (addition/removal) disabled permanently.

```
/**
 * @dev Leaves the contract without owner. It will not be possible to call
 * `onlyOwner` functions anymore. Can only be called by the current owner.
 *
 * NOTE: Renouncing ownership will leave the contract without an owner,
 * thereby removing any functionality that is only available to the owner.
 */
function renounceOwnership() public virtual onlyOwner {
    _transferOwnership(address(0));
}
```

Listing 2.8: src/DeployFactory.sol

```
modifier onlyDeployer() {
    require(isDeployer[_msgSender()], "Deployable: caller is not the deployer");
    _;
}

constructor(address _owner) {
    _transferOwnership(_owner);
    isDeployer[_owner] = true;
}

function manageDeployer(address _deployer, bool _isDeployer) external onlyOwner {
    isDeployer[_deployer] = _isDeployer;
    emit DeployerUpdated(_deployer, _isDeployer);
}

function deploy(bytes memory bytecode, bytes32 salt) external onlyDeployer returns (address) {
    address contractAddress;
    assembly {
        contractAddress := create2(0, add(bytecode, 0x20), mload(bytecode), salt)
        if iszero(extcodesize(contractAddress)) { revert(0, 0) }
    }
    emit ContractDeployed(contractAddress, salt);
    return contractAddress;
}
```

Listing 2.9: src/DeployFactory.sol

Feedback from the project The project stated that renounceOwnership should not be part of operations. The project will avoid similar operational errors.

2.2.5 Potential centralization risks

Introduced by `Version 1`

Description In this project, several privileged roles (e.g., owner, operator, and deployer) can conduct sensitive operations, which introduces potential centralization risks. For example, after the owner invokes function `renounceOwnership()`, all the previously authorized operators can still invoke `pause()`, but no one can invoke `unpause()`, which would lead to the project being permanently paused. If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.

Feedback from the project The project stated that permission management will be carefully dealed and it will not happen under normal circumstances.

2.2.6 Security audit assumptions on trusted executors

Introduced by `Version 1`

Description The current security assessment of the protocol operates under the assumption that the external routers used in the three specific executor contracts (i.e., `BGWSwapV1Executor`, `BGWSwapV1NewExecutor`, and `OneInchV6Executor`) are fully trusted. This foundational premise directly impacts all subsequent security evaluations and risk assessments conducted within the audit scope.

Feedback from the project The project stated that the executor will follow a strict whitelist management, and only the owner will have the privilege to add it.

2.2.7 Weird ERC20 tokens

Introduced by `Version 1`

Description In this protocol, there are no whitelisting restrictions on ERC20 tokens. It is important to note that some weird ERC20 tokens (e.g., Fee-on-Transfer tokens) are not supported and may result in unexpected behaviors.

Feedback from the project The project stated that this is the current design. The project will support Fee-on-Transfer tokens in other ways in the future.

2.2.8 The previous owner still retains deployer privileges after ownership is transferred

Introduced by `Version 1`

Description In contract `DeployFactory`, when the ownership has been transferred via function `transferOwnership()`, the previous owner still retains deployer privilege, which is assigned in the constructor.

```
/**
 * @dev Transfers ownership of the contract to a new account (`newOwner`).
 * Can only be called by the current owner.
 */
function transferOwnership(address newOwner) public virtual onlyOwner {
```

```
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

Listing 2.10: src/DeployFactory.sol

```
constructor(address _owner) {
    _transferOwnership(_owner);
    isDeployer[_owner] = true;
}

function manageDeployer(address _deployer, bool _isDeployer) external onlyOwner {
    isDeployer[_deployer] = _isDeployer;
    emit DeployerUpdated(_deployer, _isDeployer);
}
```

Listing 2.11: src/DeployFactory.sol

Feedback from the project The project will carefully deal with the access management.

2.2.9 Avoid mis-operations when deploying contract `DeployFactory`

Introduced by `Version 1`

Description In the `constructor` of contract `DeployFactory`, the address `_owner` is not checked to ensure it is not zero. The protocol should be careful to prevent potential mis-operations when deploying.

```
constructor(address _owner) {
    _transferOwnership(_owner);
    isDeployer[_owner] = true;
}
```

Listing 2.12: src/DeployFactory.sol

Feedback from the project The project will carefully deal with it to avoid such mis-operations when deploying the contract.

BOOST WEB3 THROUGH NEXT-GENERATION SECURITY & USABILITY INNOVATIONS